



SOFTWARE REQUIREMENTS SPECIFICATION

Student Course Recommendation and Management System

Project Team Members

Name	Student ID
Reham Ali Salah El Din	921230046
Youssef Hassan	921230162
Rouaa Ahmed Bakr	921230044
Mahmoud Hazem	921230130
Mahmoud Ahmed	921251002

1. Introduction

1.1 What Is This Document?

This document is a Software Requirements Specification (SRS) for the Student Course Recommendation and Management System (SCRMS). It explains what the system is supposed to do, who will use it, and what the main rules and conditions are.

1.2 What Is the System?

The SCRMS is a console-based academic software system built in Python. It has two main jobs:

- Data Management: It lets a user add, view, update, and delete records for students, courses, and course ratings.
- Smart Recommendations: It analyses the ratings that students have given to courses and uses that information to suggest other courses each student might enjoy.

The system stores all data in simple CSV files and keeps a log of every action the user performs. Everything runs through a text-based menu in the terminal — no website or app is needed.

2. System Scope

2.1 What the System Covers

The SCRMS handles two areas of functionality, both of which work with the same underlying data about students, courses, and ratings.

2.1.1 Management Area

This part of the system lets an authorized user manage academic records through a menu-driven console interface. The user can:

- Add, view, search, update, and delete Student records (name, email, program).
- Add, view, search, update, and delete Course records (course code, title, department, credit hours).
- Add, view, filter, update, and delete Rating records (which student gave which score to which course).
- View a history of all recent operations through the built-in log viewer.

2.1.2 Recommendation Area

This part of the system reads the stored rating data and automatically produces personalised course suggestions for each student. Specifically:

- It computes how similar courses are to each other based on how students have rated them (using cosine similarity).
- It looks at students with similar rating patterns and incorporates their preferences (collaborative filtering).
- It produces a ranked list of 5 to 10 recommended courses for each student, each with a confidence score.
- It displays the results on screen and exports them to a CSV file.

2.2 Data the System Works With

Entity	What It Represents	Key Fields
Student	A registered university student.	ID, Full Name, Email, Academic Program
Course	An academic course offered by the university.	ID, Course Code, Title, Department, Credit Hours
Rating	A numerical score (1.0–5.0) a student gave to a course.	ID, Student ID, Course ID, Score, Timestamp

2.3 How Data Is Stored

All data is saved in CSV (Comma-Separated Values) files. These are plain text files that can be opened in any spreadsheet program like Microsoft Excel or Google Sheets. There are three data files (students.csv, courses.csv, ratings.csv) and one log file (system_operations.log).

3. User Roles

3.1 Overview

The system is designed for three types of users:

User Role	Who They Are	What They Do	Technical Level
Academic Administrator	A university staff member responsible for maintaining academic records.	Uses the console menu to add, update, search, and delete students, courses, and ratings. Also runs the recommendation pipeline and views the operation log.	Basic - needs only basic comfort with typing in a terminal.
Student (indirect)	A university student whose data and ratings are stored in the system.	Does not interact with the system directly. Receives the output of the recommendation process via the exported CSV file or console display.	None - the student never opens or operates the system.
System Developer / Evaluator	The engineering student who built the system, or the instructor who evaluates it.	Has full access to the system for the purposes of testing, demonstration, and academic review.	High - understands the full codebase and architecture.

3.2 What Each Role Needs

3.2.1 Academic Administrator

- A clear, numbered menu that guides them through every available action.
- Plain-English error messages that tell them exactly what went wrong and how to fix it.
- A confirmation step before any record is permanently deleted.
- Immediate saving of all changes so nothing is lost if the program is closed.
- The ability to trigger the recommendation process and receive results quickly.

3.2.2 Student (Indirect User)

- Recommendations that genuinely reflect their academic history and preferences.
- No recommended courses that they have already rated.
- Recommendations presented in a clear format that is easy to read and act upon.

3.2.3 System Developer / Evaluator

- A clean, well-commented, object-oriented codebase that is easy to read and review.
- A working unit test that demonstrates correctness.
- Consistent, reproducible results on the same input data.

3.3 Stakeholders

Stakeholder	Interest
Course Instructor	Ensuring the system effectively manages academic records while producing accurate recommendations.
Student Developers(us)	Producing a system that satisfies all stated requirements and demonstrates strong engineering practice.
Enrolled Students (hypothetical)	Receiving accurate, useful course suggestions based on their personal academic history.
Future Maintainers	Receiving a codebase that is clean, well-documented, and easy to extend or modify.

4. Functional Requirements

4.1 System Initialization

- **[FR-01]** The system shall load all existing student, course, and rating records from the CSV data files into memory when it starts.
- **[FR-02]** The system shall automatically create empty data files with the correct column headers if no files are found on start-up.
- **[FR-03]** The system shall record a start-up event in the operation log, including the date and time and the number of records loaded for each entity type.
- **[FR-04]** The system shall display a welcome message and the main menu immediately after successful start-up.
- **[FR-05]** If an existing data file cannot be read (due to a formatting error for example), the system shall display a clear error message and stop without deleting or overwriting the file.

4.2 Student Management

- **[FR-06]** The system shall allow the user to add a new student by entering a full name, a university email address, and an academic program. The system shall automatically assign a unique Student ID.
- **[FR-07]** The system shall reject a student name that contains numbers or special characters, and display an error message explaining the correct format.
- **[FR-08]** The system shall reject an email address that does not follow the format name@domain.extension, and display an error message.
- **[FR-09]** The system shall reject a new student if the email address is already used by another student in the system.
- **[FR-10]** The system shall allow the user to view all students in a neat table showing ID, name, email, and program.
- **[FR-11]** The system shall allow the user to search for a student by their ID and display that student's full details.
- **[FR-12]** The system shall allow the user to update a student's name, email, or program. The Student ID cannot be changed.
- **[FR-13]** The system shall allow the user to delete a student. Before deleting, the system shall warn the user how many rating records will also be removed and ask for confirmation by typing 'YES'.
- **[FR-14]** When a student is deleted, the system shall also automatically delete all rating records associated with that student.
- **[FR-15]** Every add, update, and delete action on a student record shall be saved to the CSV file immediately and recorded in the log.

4.3 Course Management

- **[FR-16]** The system shall allow the user to add a new course by entering a course code, title, department, and number of credit hours. The system shall automatically assign a unique Course ID.
- **[FR-17]** The system shall reject a course code that is already used by another course in the system.
- **[FR-18]** The system shall reject a credit hours value that is not a whole number between 1 and 3.
- **[FR-19]** The system shall allow the user to view all courses in a neat table showing ID, code, title, department, and credit hours.
- **[FR-20]** The system shall allow the user to search for a course by its ID or its course code.
- **[FR-21]** The system shall allow the user to update a course's title, department, or credit hours. The course code cannot be changed.
- **[FR-22]** The system shall allow the user to delete a course, with the same confirmation and cascade-deletion behavior as student deletion (FR-13, FR-14).
- **[FR-23]** Every add, update, and delete action on a course record shall be saved to the CSV file immediately and recorded in the log.

4.4 User Access Control

- **[FR-24] User Identification:** The system shall require the user to enter a unique ID upon start-up to determine their access level (Administrator or Student).
- **[FR-25] Restricted Student View:** If a Student ID is entered, the system shall restrict access to administrative functions (like viewing all records in FR-10) and only permit personal record viewing and recommendation generation.

4.5 Rating Management

- **[FR-26]** The system shall allow the user to record a rating by choosing a Student ID, a Course ID, and entering a score. The system shall automatically assign a unique Rating ID and record the date and time.
- **[FR-27]** The system shall reject a rating if the Student ID does not match any existing student.
- **[FR-28]** The system shall reject a rating if the Course ID does not match any existing course.
- **[FR-29]** The system shall reject a rating score that is not a number between 1.0 and 5.0.
- **[FR-30]** If the same student has already rated the same course, the system shall not create a duplicate. Instead, it shall inform the user and offer the option to update the existing score.

- **[FR-31]** The system shall allow the user to view all ratings in a table, with options to filter by Student ID or Course ID.
- **[FR-32]** The system shall allow the user to update the score of an existing rating by its Rating ID.
- **[FR-33]** The system shall allow the user to delete a specific rating by its Rating ID.
- **[FR-34]** Every add, update, and delete action on a rating record shall be saved to the CSV file immediately and recorded in the log.

4.5 Recommendation Engine

- **[FR-35]** The system shall skip and report any invalid rating records encountered during loading (e.g. missing fields or out-of-range scores), and continue processing the rest.
- **[FR-36]** The system shall load the stored rating data and build a student-course rating table (matrix) in memory when the recommendation process is started.
- **[FR-37]** The system shall calculate how similar each pair of courses is, based on the scores students have given them, using the cosine similarity formula.
- **[FR-38]** If a course has received no ratings at all (and therefore has a zero rating vector), the system shall assign it a similarity score of 0.0 and continue without crashing.
- **[FR-39]** The system shall also calculate how similar each pair of students is to each other, based on their rating patterns, to support collaborative filtering.
- **[FR-40]** For each student, the system shall identify all courses that student has not yet rated and calculate a recommendation score for each one.
- **[FR-41]** The recommendation score for each unrated course shall be based on a combination of item-based similarity (how similar that course is to courses the student liked) and user-based collaborative filtering (what courses similar students liked).
- **[FR-42]** The system shall produce a ranked recommendation list of between 5 and 10 courses for each student. If fewer than 5 unrated courses are available, all available ones shall be recommended.
- **[FR-43]** The system shall not recommend any course the student has already rated.
- **[FR-44]** Each recommendation shall include a confidence score between 0.0 and 1.0 showing how strong the recommendation is.
- **[FR-45]** The system shall display all recommendations clearly on the console, showing each student's name, ID, and their ranked list of recommended courses with confidence scores.
- **[FR-46]** The system shall export all recommendation results to a CSV file named with the date and time .=
- **[FR-47]** After exporting, the system shall display the full file path of the saved CSV file.

4.6 Operation Log

- **[FR-48]** The system shall maintain a log file (system_operations.log) that records every action performed.

- **[FR-49]** Each log entry shall include: the date and time, the type of action (ex: ADD, UPDATE, DELETE), the type of record affected (Student, Course, or Rating), the record's ID, and whether the action succeeded or failed.
- **[FR-50]** The system shall also log start-up and shutdown events, and every time a user input fails validation.
- **[FR-51]** The user shall be able to view the most recent log entries through the console menu, displayed from newest to oldest. The number of entries shown can be set by the user (default: 20).
- **[FR-52]** The log file shall never be cleared or modified by the system. New entries are always added to the end of the file.

4.7 Menu Navigation and Shutdown

- **[FR-53]** The system shall display a main menu with the following options: (1) Manage Students, (2) Manage Courses, (3) Manage Ratings, (4) Run Recommendations, (5) View Log, (6) Exit.
- **[FR-54]** Each management menu shall offer the options: Add, View All, Search, Update, Delete, and Return to Main Menu.
- **[FR-55]** After every completed action, the system shall return to and re-display the current menu automatically.
- **[FR-56]** If the user enters an invalid menu choice (e.g.: a letter instead of a number), the system shall display 'Invalid option. Please enter a number from the list.' and show the menu again.
- **[FR-57]** When the user chooses Exit, the system shall confirm all data is saved, write a shutdown entry to the log, and close cleanly.

5. Non-Functional Requirements

5.1 Performance

- **[NFR-01]** The system shall finish loading all data and display the main menu within 5 seconds for a dataset of up to 10,000 total records.
- **[NFR-02]** Any single add, update, or delete action shall complete and return to the menu within 3 seconds for datasets of up to 10,000 records.
- **[NFR-03]** The full recommendation process shall be completed in less than 5 seconds for the specified dataset.
- **[NFR-04]** The system shall maintain minimal resource usage, ensuring stable performance on standard student laptops.

5.2 Reliability

- **[NFR-05]** The system shall never crash or freeze because of invalid user input. Any bad input shall be caught, an error message shown, and the menu re-displayed.
- **[NFR-06]** Every confirmed save action shall write data to the CSV file before control returns to the menu, so no confirmed change is ever lost during normal use.
- **[NFR-07]** Running the recommendation process on the same input data with the same settings shall always produce exactly the same results.
- **[NFR-08]** The system shall handle edge cases gracefully, including: an empty dataset, a student who has rated every available course, and a course that has never been rated by any student.

5.3 Usability

- **[NFR-09]** A user who is comfortable using a terminal but has never seen this system before shall be able to complete any management task using only the on-screen menus, without needing a manual.
- **[NFR-10]** Every error message shall be written in plain English, name the specific field that caused the problem, and explain what the correct input looks like.
- **[NFR-11]** Every successful action shall be followed by a clear confirmation message stating what was done and the ID of the affected record.
- **[NFR-12]** Any delete action shall require the user to type 'YES' to confirm. The prompt shall clearly state what will be deleted, including how many related records will also be removed.
- **[NFR-13]** For any operation that takes longer than one second, the system shall display a progress message so the user knows it is still working.

5.4 Maintainability

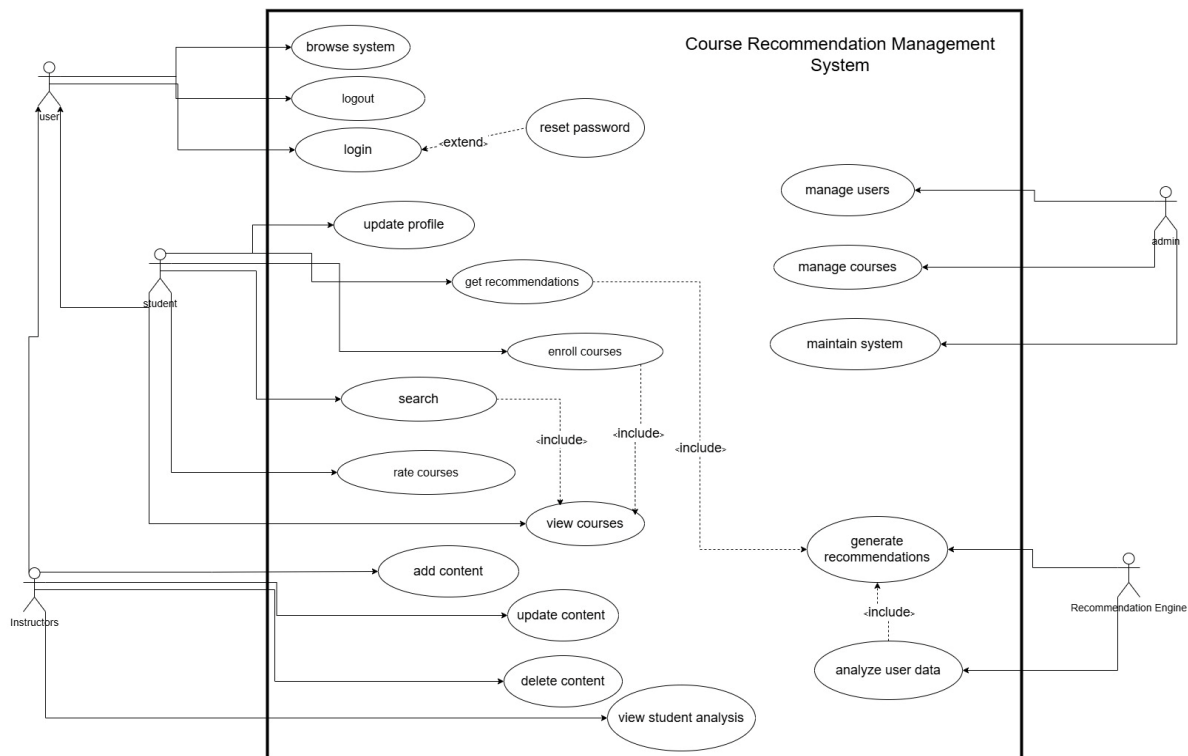
- **[NFR-14]** All source code shall follow PEP 8 Python coding style guidelines (naming, indentation, line length, etc.).
- **[NFR-15]** Every class and every public method shall have a docstring that describes what it does, what inputs it expects, and what it returns.
- **[NFR-16]** The code shall be divided into separate modules by responsibility: one for data models, one for data access, one for business logic, one for the recommendation pipeline, and one for the user interface. No module shall depend on one above it in this hierarchy.
- **[NFR-17]** All settings that might need to change (such as file paths, the number of recommendations to show, or validation limits) shall be defined in a single configuration file, not scattered throughout the code.

- **[NFR-18]** The project shall include a unit test suite that covers at least 80% of the code in all non-trivial modules.

5.5 Security

- **[NFR-19]** The system shall implement role-based access control. Both Administrator and Student roles shall require unique passwords to access their respective functionalities and ensure data privacy.
- **[NFR-20]** The user shall not be able to delete all records of any type in one action. Every deletion must target one specific record at a time.
- **[NFR-21]** The system shall never write data outside of its own project folder. Any file paths entered by the user shall be checked to prevent access to other parts of the file system.
- **[NFR-22]** The system shall not use Python's `eval()` or `exec()` functions on any data entered by the user.
- **[NFR-23]** The log file shall always be opened in append-only mode. No part of the system shall be able to delete or overwrite existing log entries.

6.1 Use Case Diagram



6.2 Use Case Specifications

1. Use Case: Manage users

- **Actor:** Admin
- **Description:** Allows the admin to perform CRUD (Create, Read, Update, Delete) operations on student and instructor records.
- **Trigger:** Admin selects the "Manage Users" option from the main menu.
- **Pre-conditions:** The system has successfully initialized and loaded students.csv/instructors.csv.
- **Post-conditions:** The students.csv file/ instructors.csv and system logs are updated with the changes.

Basic Flow:

1. **Admin chooses an action:** Add, View, Search, Update, or Delete.
2. **If Add:** Admin enters Name, Email, and Program; system validates format and assigns a unique ID.
3. **If Delete:** System calculates linked rating records and requires "YES" confirmation before removal.

4. System saves changes immediately to CSV and returns to the menu.

2. Use Case: Manage Courses

- **Actor:** Admin
- **Description:** Enables the admin to manage the university's course catalog (Add, Update, Search, Delete).
- **Trigger:** Admin selects the "Manage Courses" option from the main menu.
- **Pre-conditions:** The system is running and courses.csv is accessible.
- **Post-conditions:** Courses are updated in the persistent CSV storage and the operation log.
- **Basic Flow:**
 1. Admin selects a course management action from the sub-menu.
 2. **If Add:** Admin enters Code, Title, Department, and Credit Hours (1-3).
 3. **If Search:** Admin enters Course ID or Course Code to retrieve details.
 4. The system validates inputs, updates the record, and records the action in system_operations.log.

3. Use Case: Rate Courses

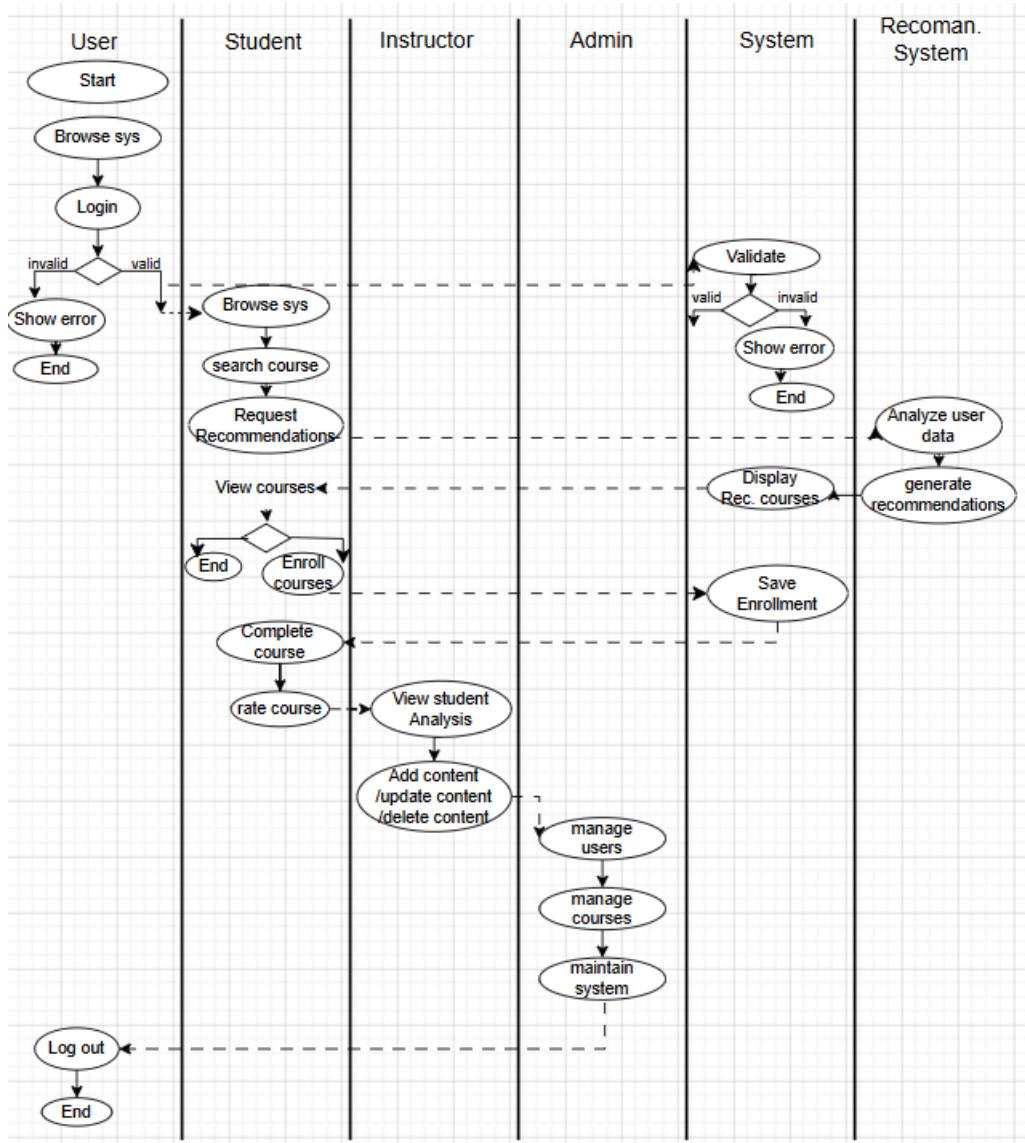
- **Actor:** Student
- **Description:** Records a numerical score given by a student to a specific course to build the recommendation dataset.
- **Trigger:** User selects "Manage Ratings" or "Rate Courses".
- **Pre-conditions:** Both the Student ID and Course ID must already exist in the system.
- **Post-conditions:** A new rating record is created in ratings.csv.
- **Basic Flow:**
 1. User enters Student ID and Course ID.
 2. User enters a score between 1.0 and 5.0.
 3. System checks if a rating already exists for this pair; if so, it can not be updated.
 4. System assigns a Rating ID and timestamp, then saves the data.

4. Use Case: Generate Recommendations

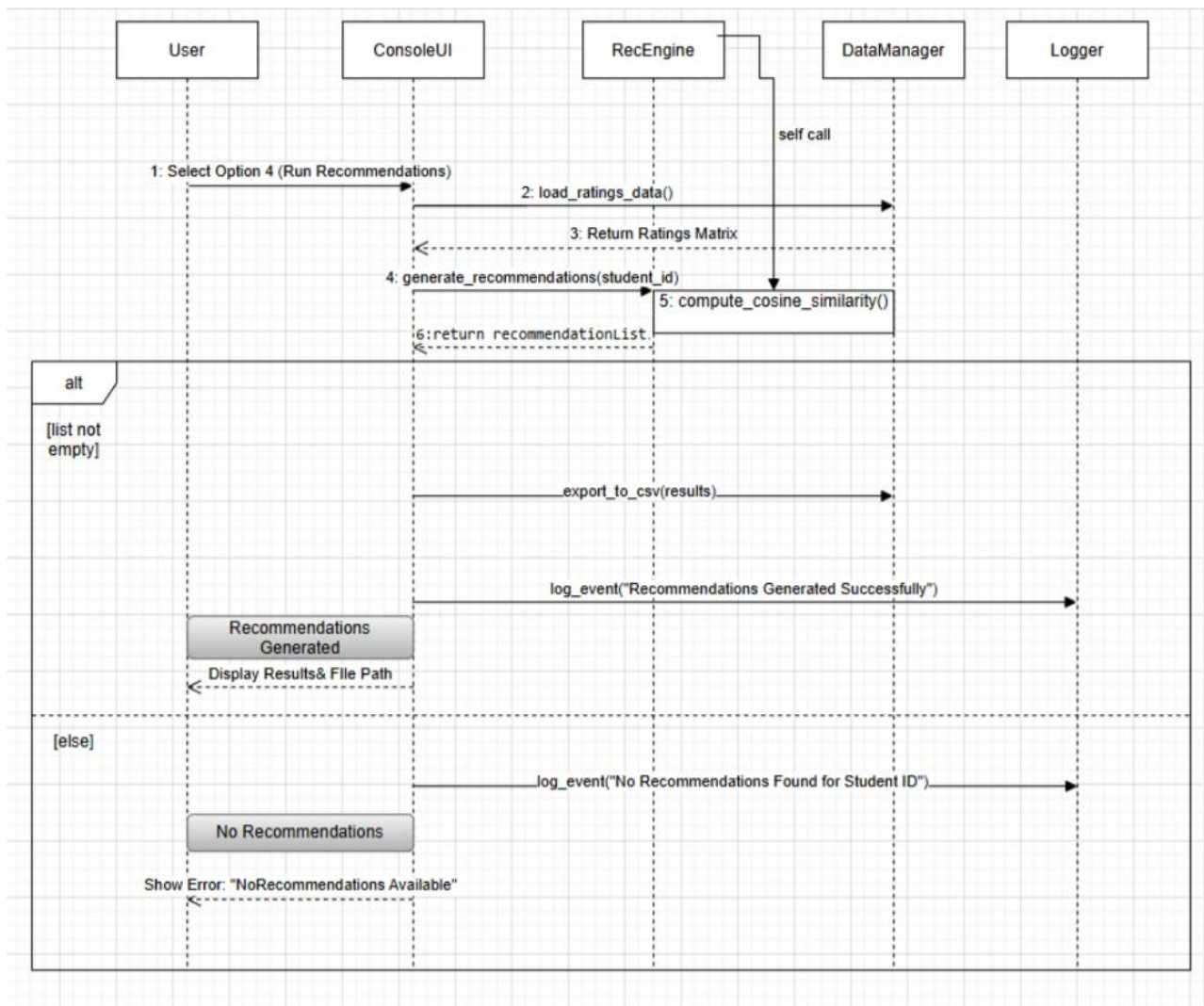
- **Actor:** Recommendation Engine
- **Description:** Processes student-course rating data to predict and suggest 5–10 new courses for a student.
- **Trigger:** User selects "Run Recommendations".
- **Pre-conditions:** Rating data must be loaded into memory and processed into a similarity matrix.
- **Post-conditions:** Recommendations are displayed on the console and exported to a timestamped CSV file.
- **Basic Flow:**

1. System calculates course and student similarities using cosine similarity and collaborative filtering.
2. System identifies courses the student has not yet rated.
3. A ranked list of recommendations with confidence scores is generated.
4. The system displays results.

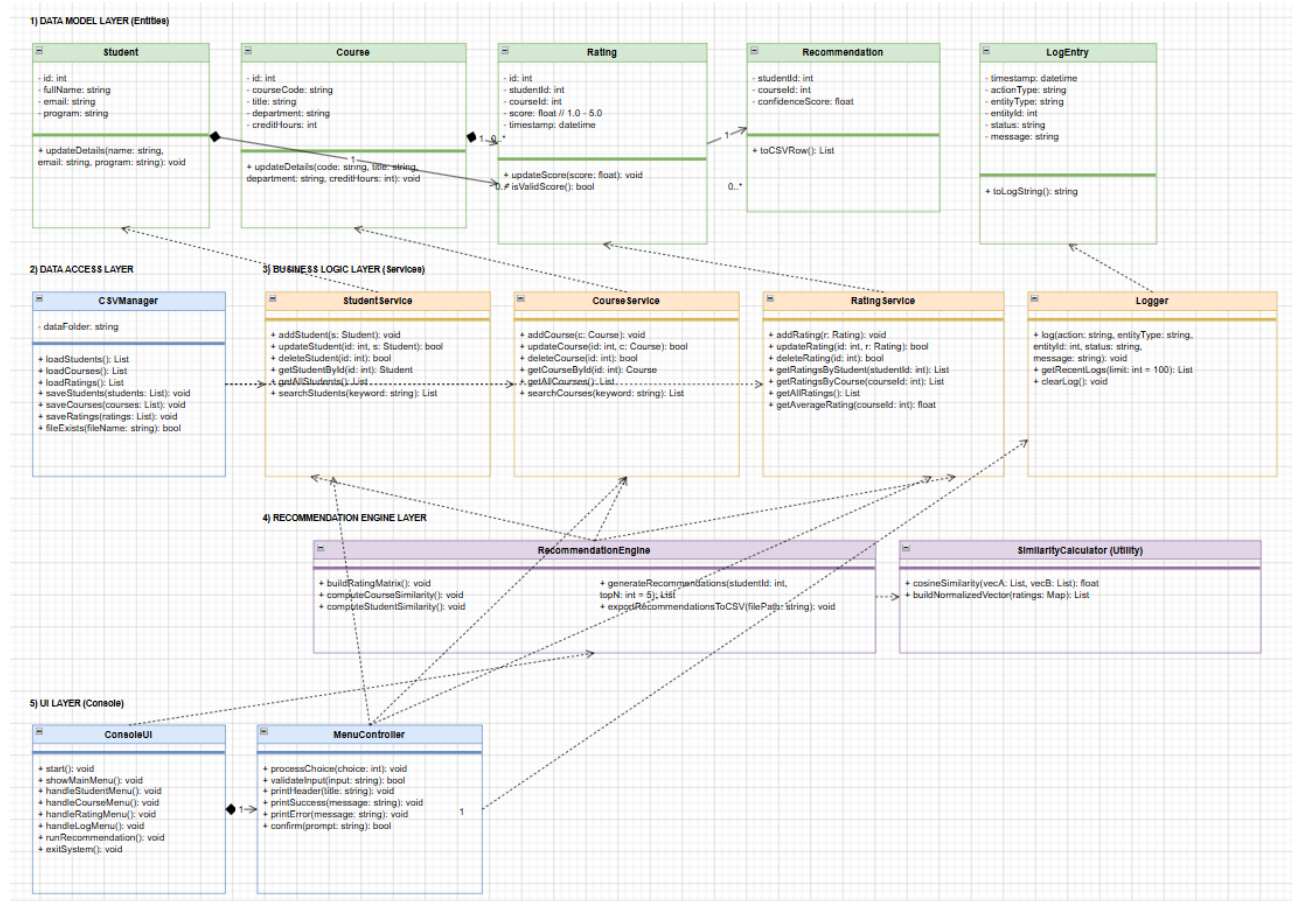
7.1 Activity Diagram



8.1 Sequence Diagram

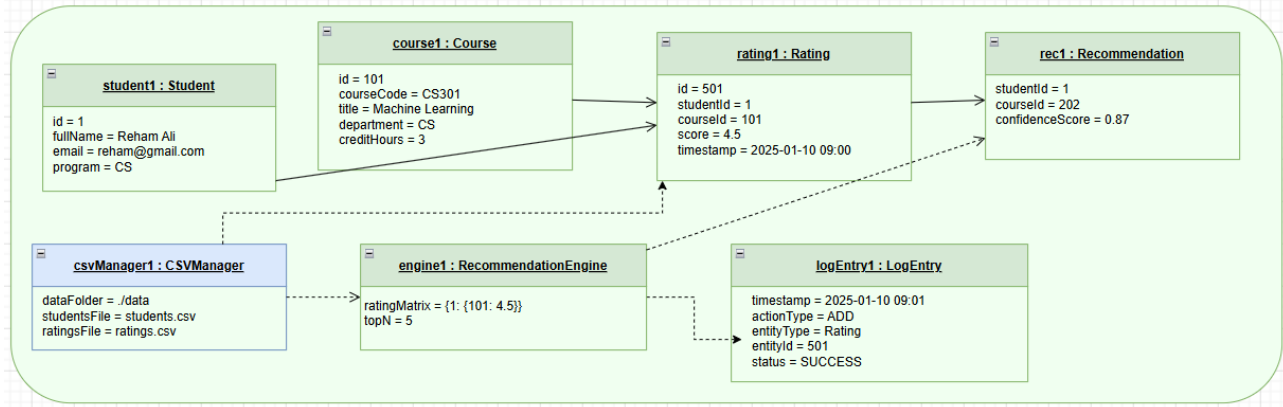


9.1 Class Diagram



9.2 Object Diagram

Scenario 1: Student rates a course and gets recommendations

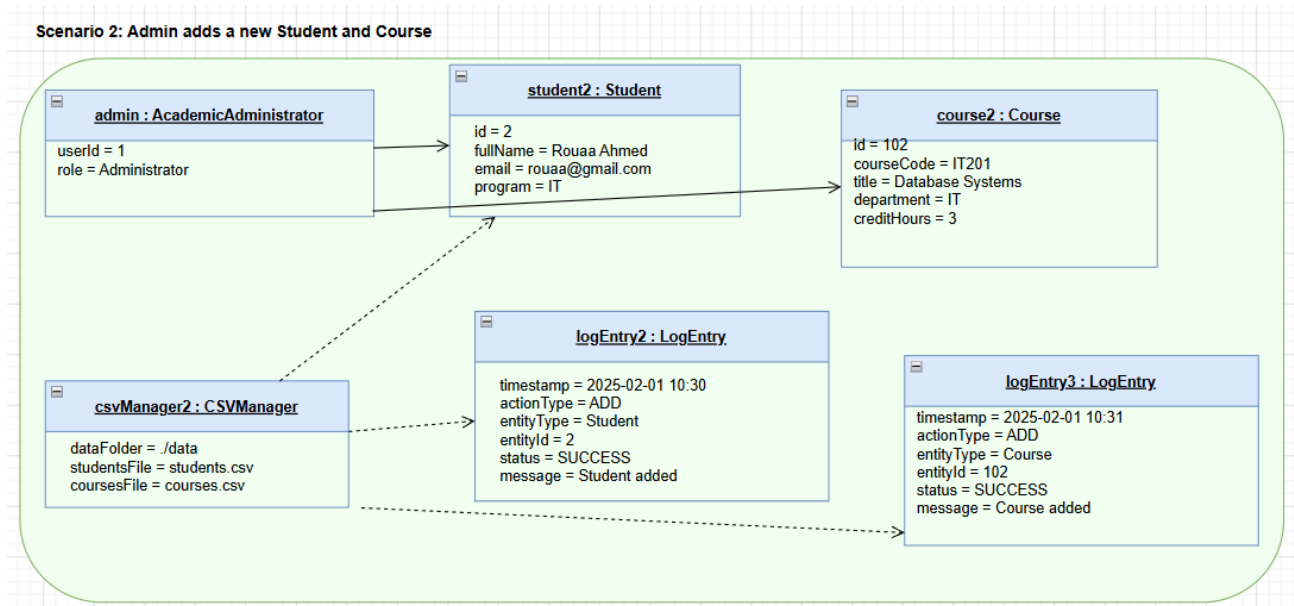


This object diagram represents a concrete runtime snapshot of when a student rates a course and the system generates recommendations.

Objects and Their Values

- **student1** is a runtime instance of the Student class representing Reham Ali, a Computer Science student with the university email reham@gmail.com. This is the actual student whose interaction triggers the entire recommendation flow.
- **course1** is a runtime instance of the Course class representing Machine Learning (CS301), a 3-credit-hour course in the CS department. This is the course being rated.
- **rating1** is a runtime instance of the Rating class created when Reham rates Machine Learning with a score of 4.5 out of 5.0, recorded at 2025-01-10 09:00. It links student1 to course1 through their IDs.
- **csvManager1** is a runtime instance of CSVManager responsible for persisting all data to the local CSV files students.csv and ratings.csv stored in the data folder.
- **engine1** is a runtime instance of RecommendationEngine that reads the rating matrix {1: {101: 4.5}} built from the stored ratings and computes cosine similarity to generate recommendations with a topN value of 5.
- **rec1** is a runtime instance of Recommendation produced by engine1, suggesting course 202 to student 1 with a confidence score of 0.87, meaning the system is 87% confident this course matches the student's academic preferences.

- **logEntry1** is a runtime instance of LogEntry automatically created after the rating is saved, recording the action type as ADD, entity type as Rating, entity ID as 501, and status as SUCCESS at 2025-01-10 09:01.

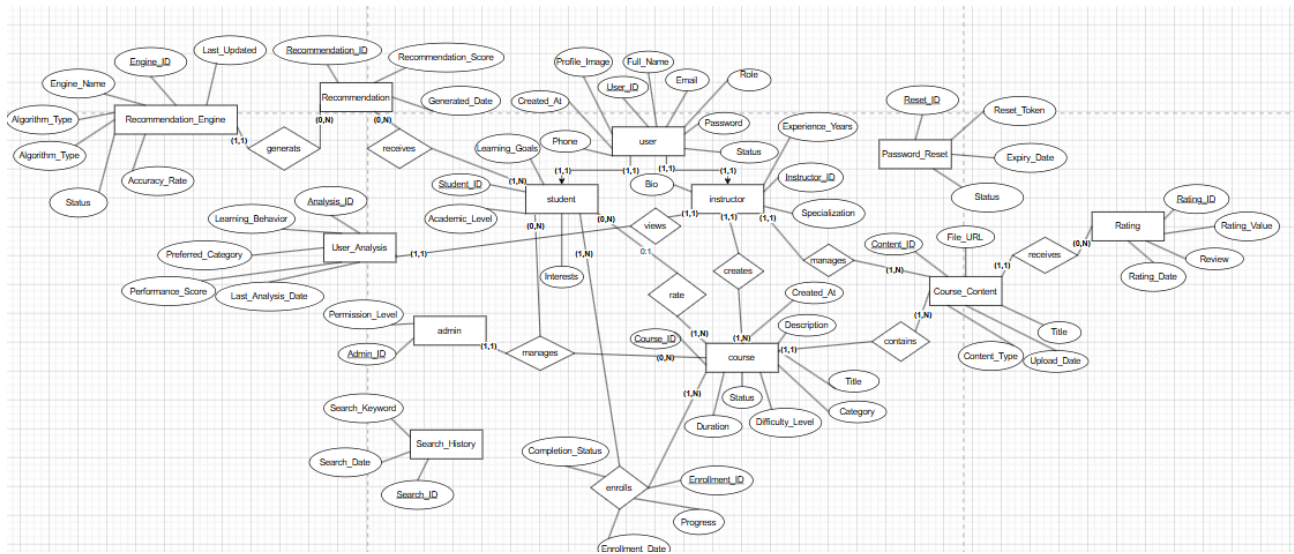


This object diagram represents a runtime snapshot of the SCRMS during an administrative CRUD operation.

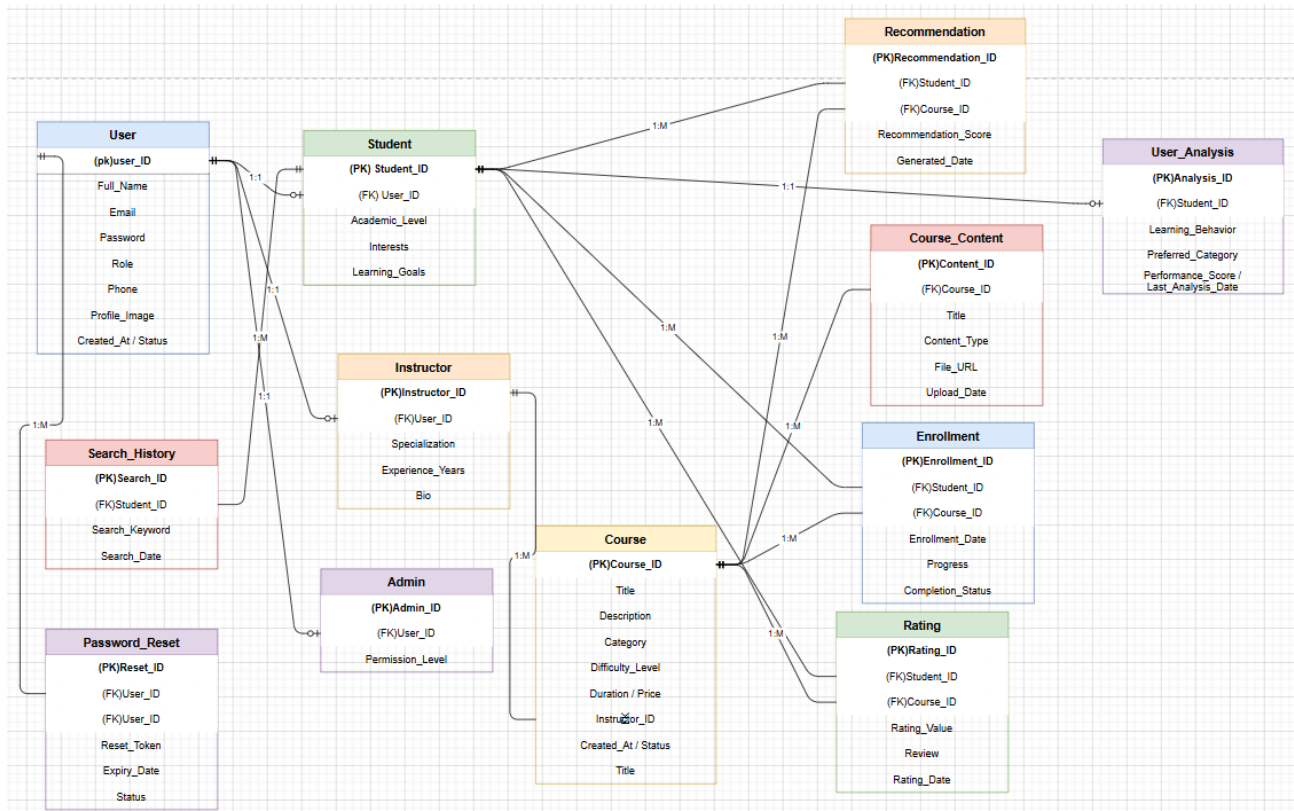
Objects and Their Values

- **admin** is an instance of AcademicAdministrator with role Administrator the only actor authorized to modify system records.
- **student2** is a new Student instance for Rouaa Ahmed, an IT student added by the admin. **course2** is a new Course instance for Database Systems (IT201), also added by the admin in the same session.
- **csvManager2** is the CSVManager instance responsible for persisting both records saving student2 to students.csv and course2 to courses.csv immediately after each addition.
- **logEntry2** and **logEntry3** are two separate LogEntry instances automatically generated after each save operation, recording the ADD action for the Student and Course respectively, both with status SUCCESS and one-minute apart timestamps.

10.1 ER Diagram

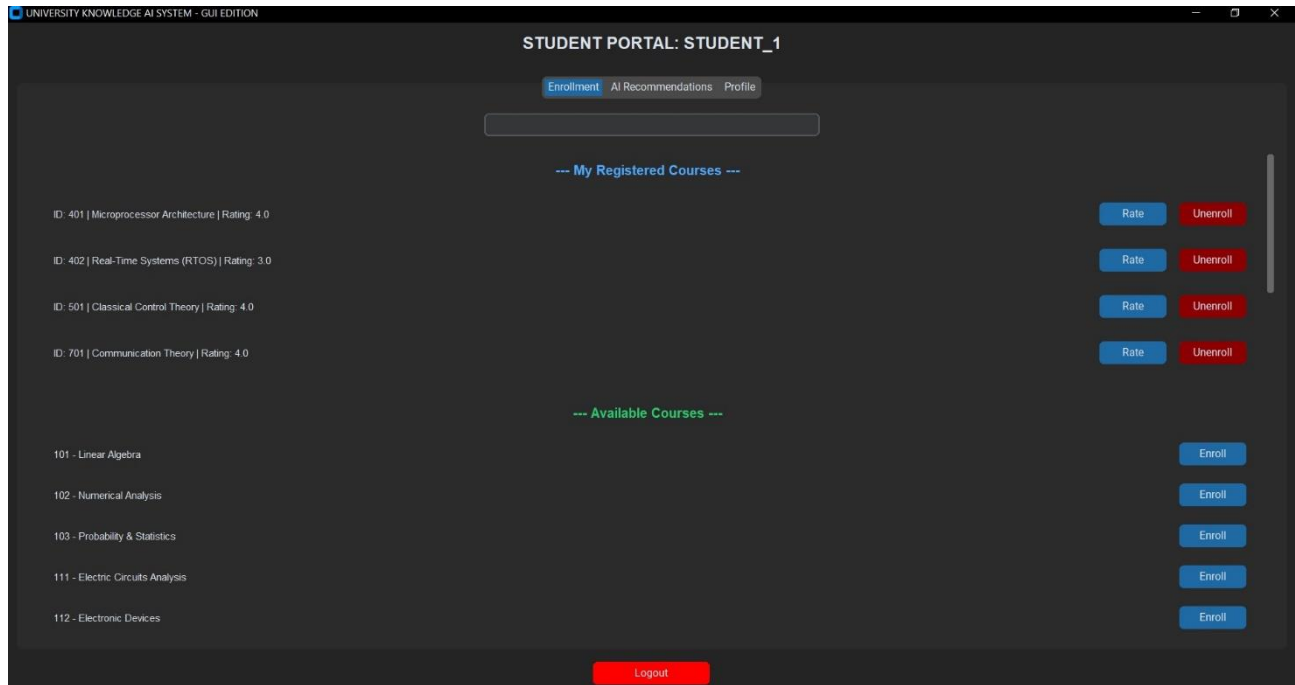


10.2 Relational Model

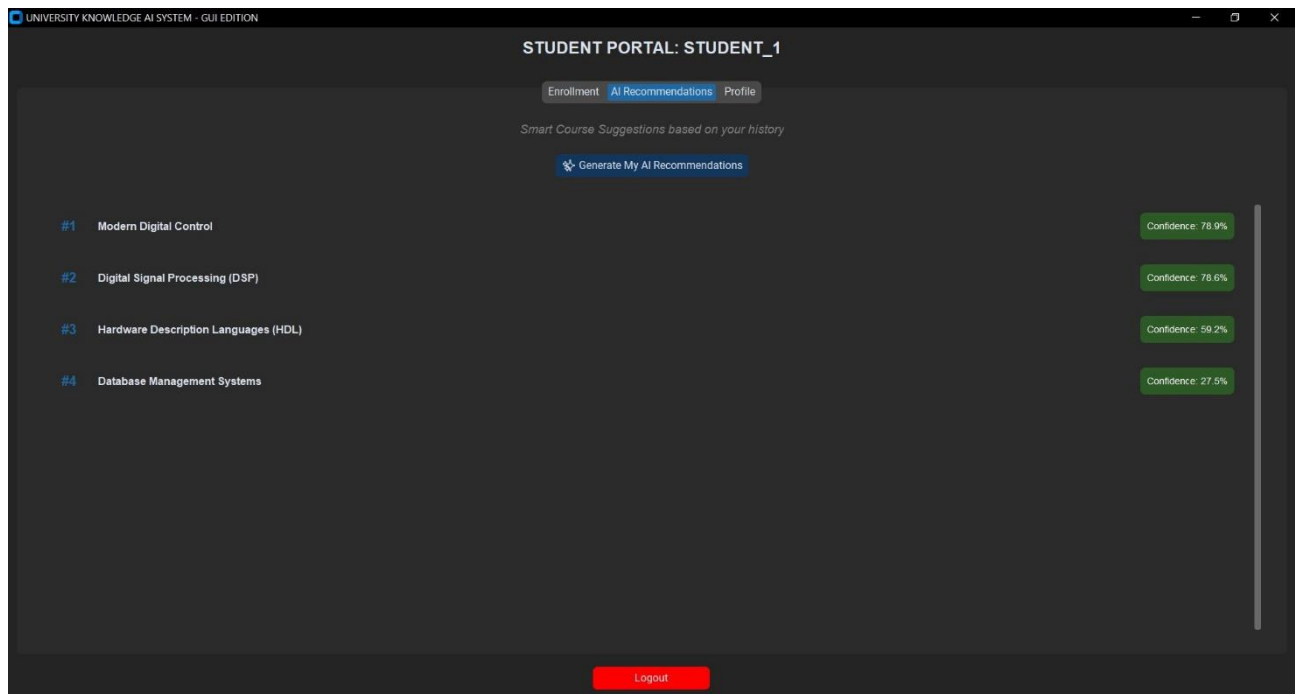


11 Visual Documentation

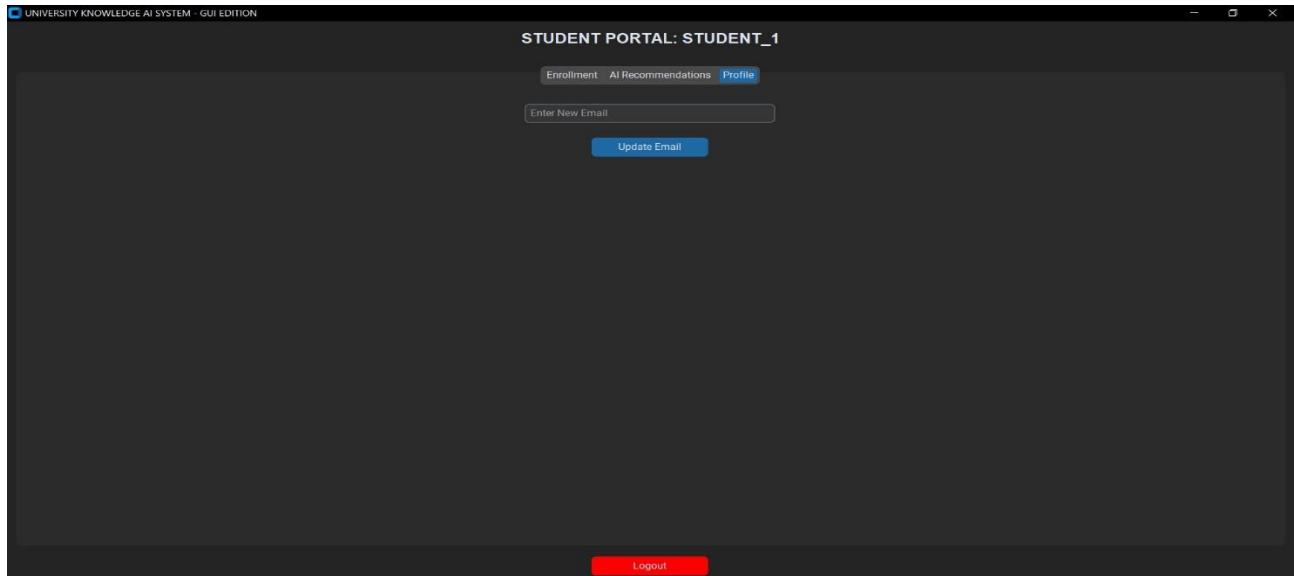
11.1 Student Portal



Student enrollment page for course registration and rating.

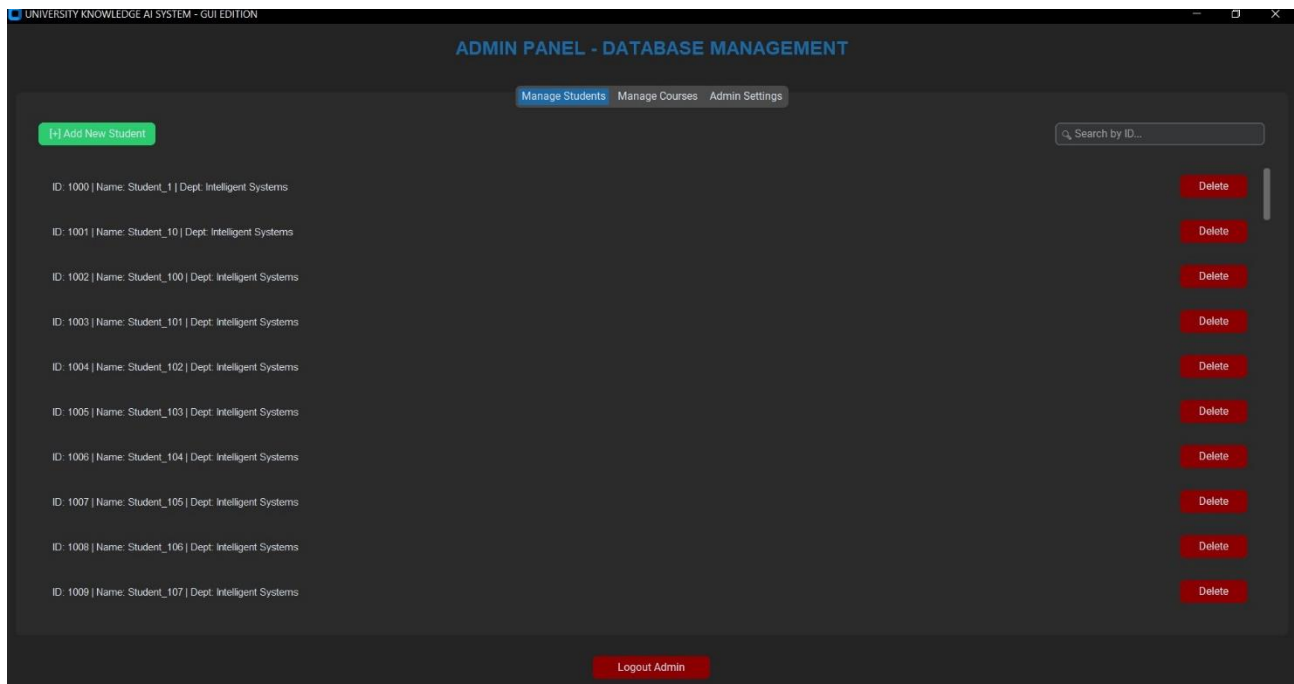


AI-powered course recommendation system based on student history.

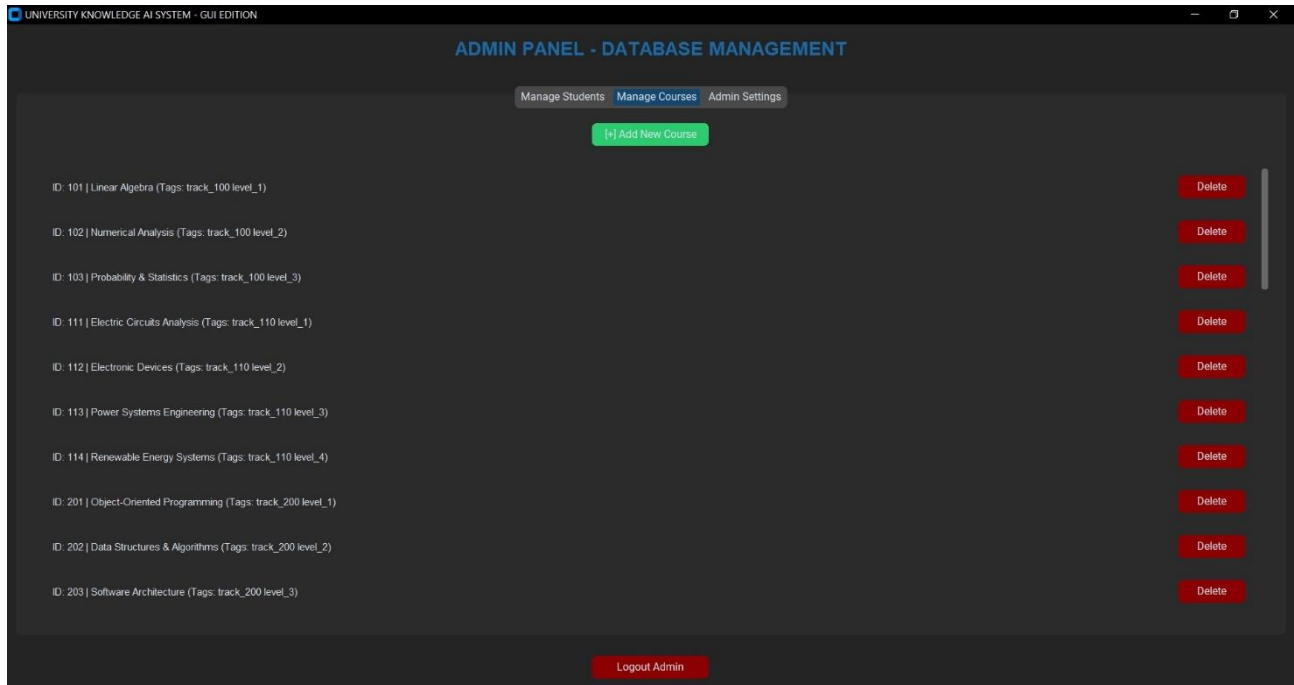


Personal profile settings for students to update their information.

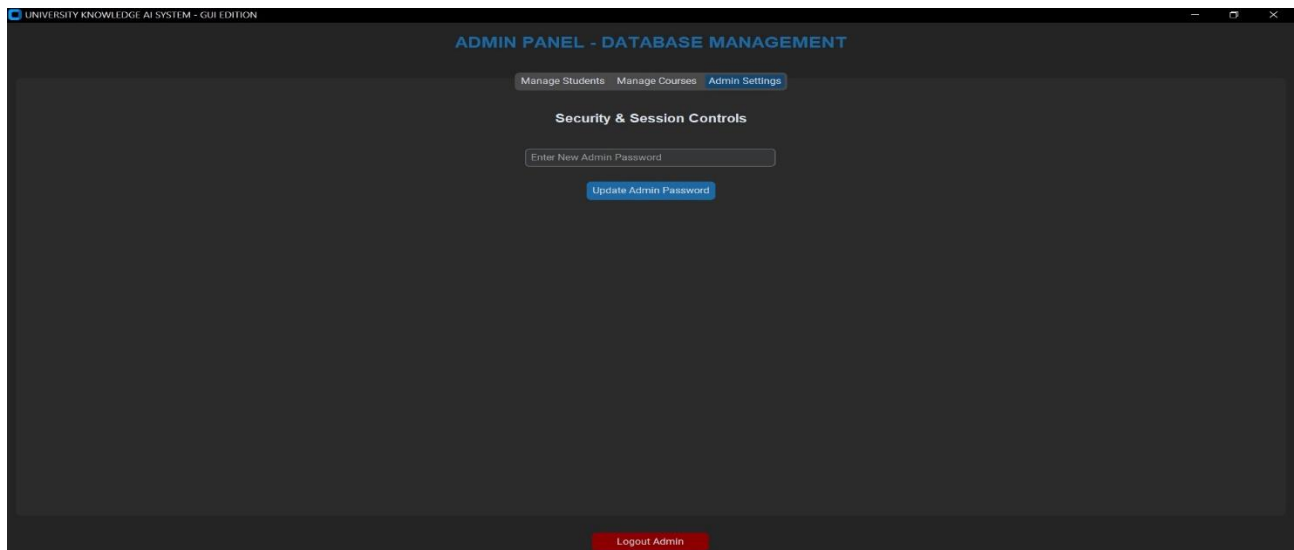
11.2 Admin Panel



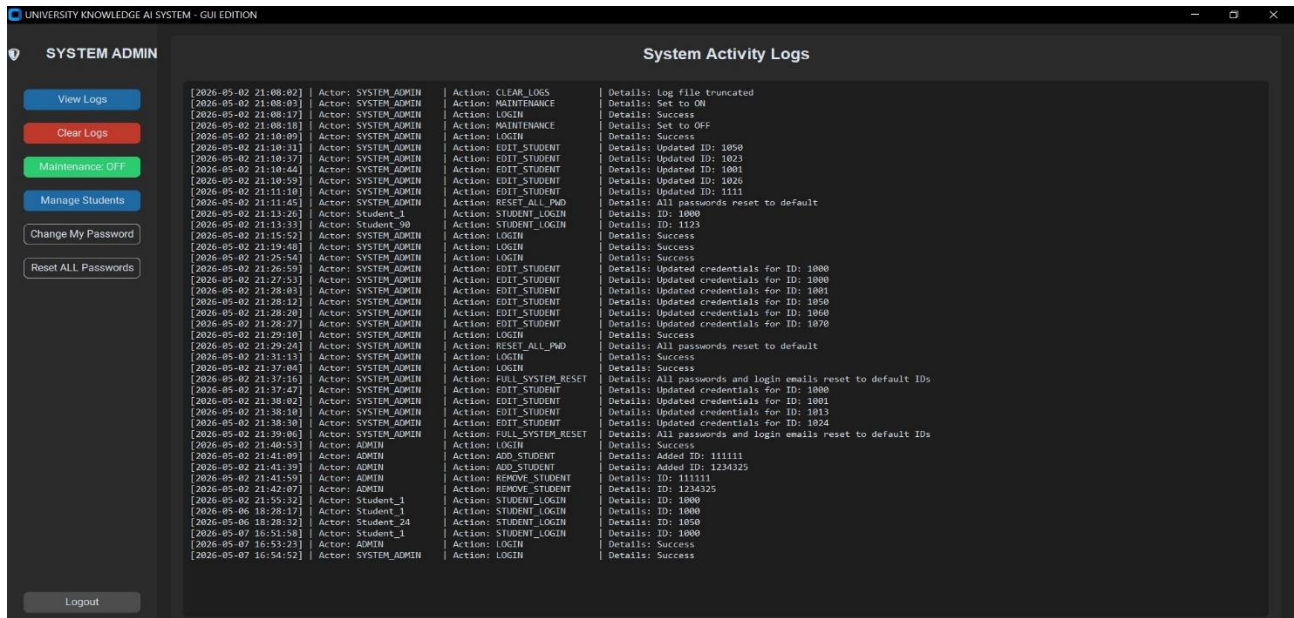
Admin dashboard for managing and searching student records.



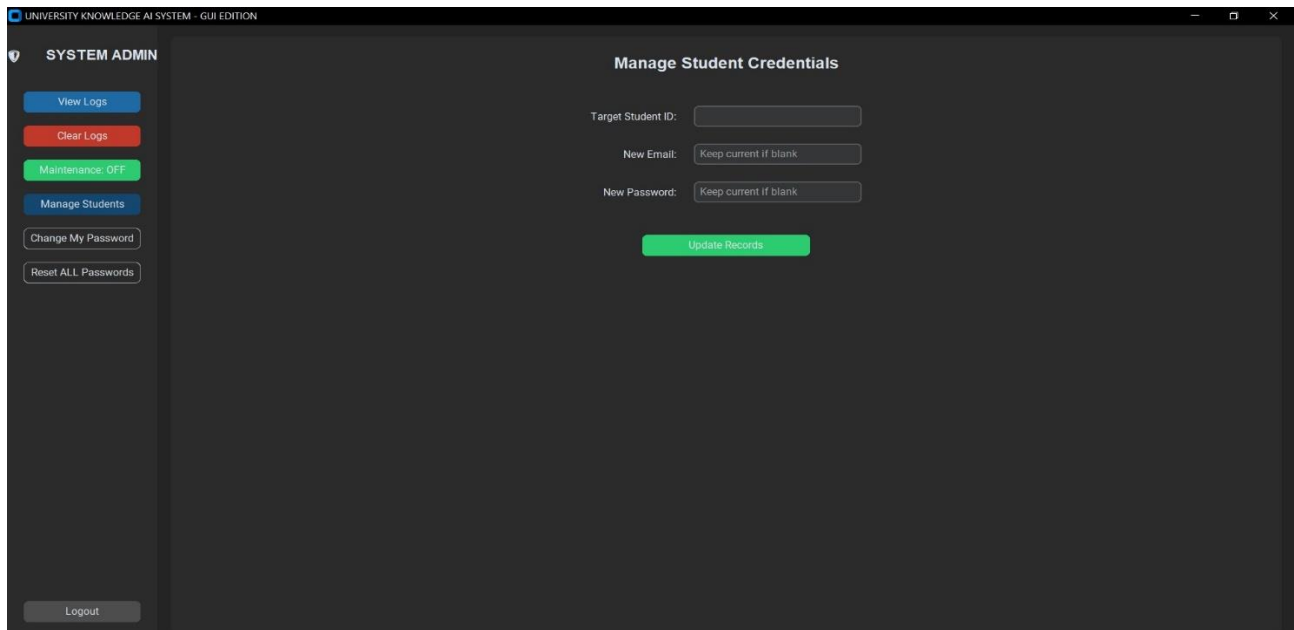
Course management interface to add, edit, or delete courses.



Admin security settings and session control panel.

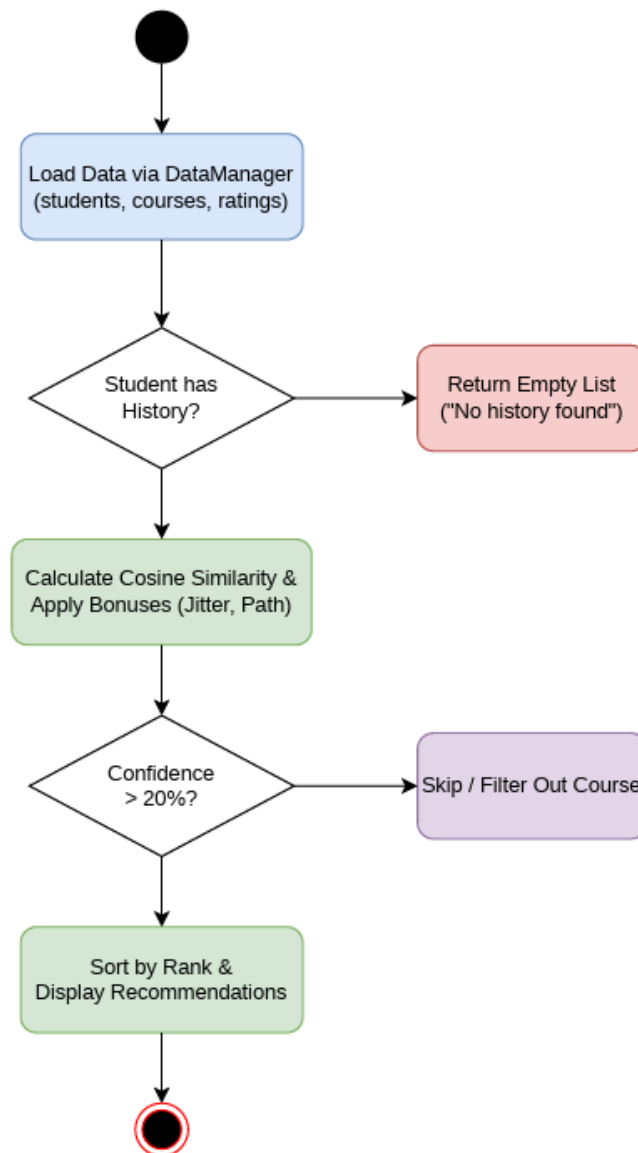


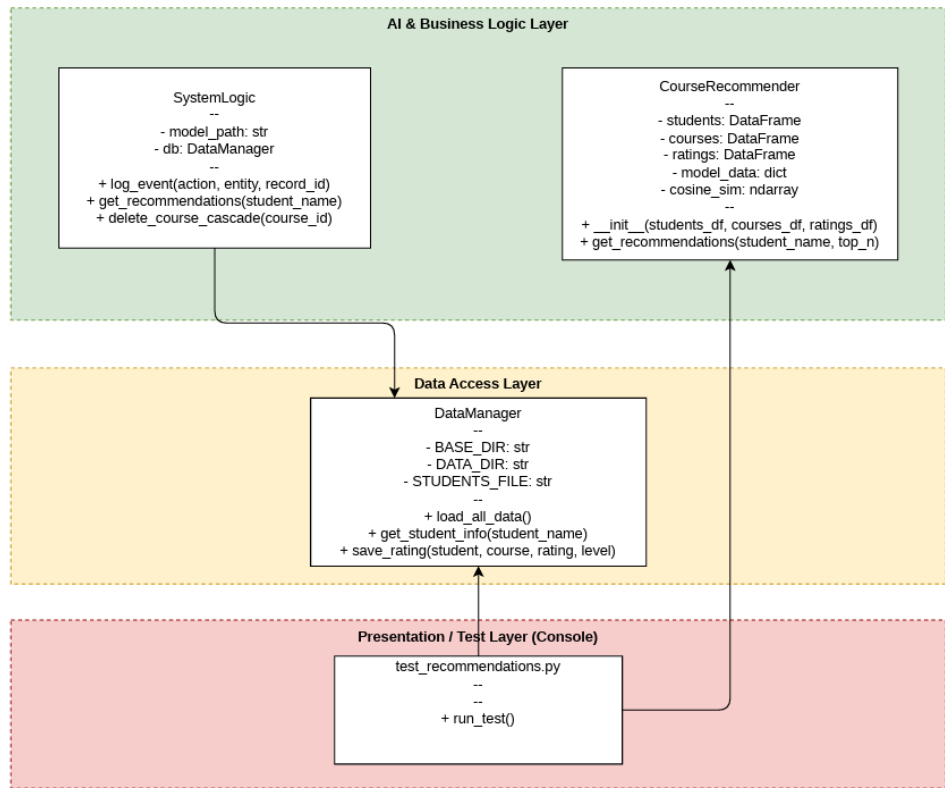
System logs to monitor all administrative and user activities.



Tool for updating or resetting student login credentials.

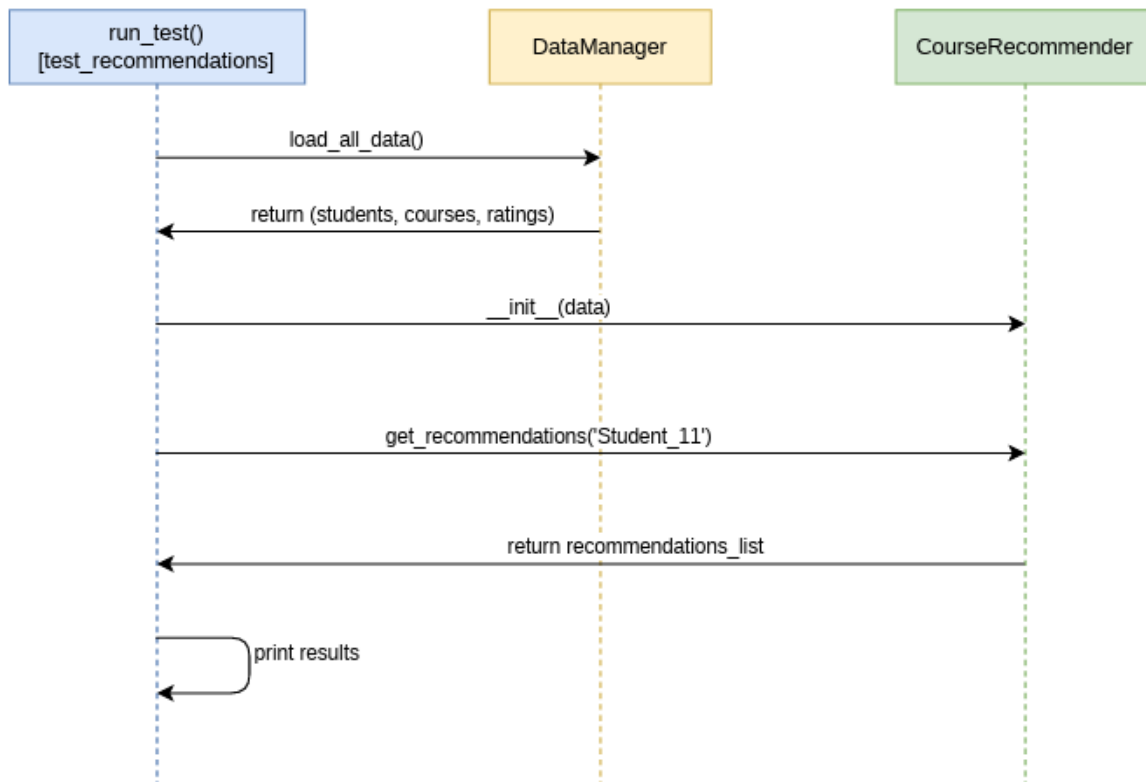
12.1 Testing





System Logic & AI Test Cases (Traceability Matrix)

ID	Feature / Logic	Input Data / Action	Expected Result	Status
TC-01	Confidence Filter	Student_11, threshold = 20%	Only courses with final_conf >= 20.0 are returned	PASSED
TC-02	Deterministic Jitter	Call get_recommendations twice for same student	Results are identical (Random seed locks jitter)	PASSED
TC-03	Cascade Delete	delete_course_cascade(101)	Course removed from courses.csv & ratings.csv + logged	PASSED
TC-04	Model Missing	Rename/Delete .pkl, call get_recommendations	Returns: 'Model file not found. Please train...'	PASSED



Testing & Validation Requirements

1. Test Scenarios

The following test scenarios cover the core logic of the Course Recommendation System, validated across system logic, AI filtering, data integrity, and error handling layers. All four scenarios were executed and passed.

ID	Feature / Logic	Input Data / Action	Expected Result	Layer	Status
TC-01	Confidence Filter	Student_11, threshold = 20%	Only courses with final_conf >= 20.0 are returned	AI Logic	PASSED
TC-02	Deterministic	Request recommendations twice for the same student	Results are identical (Random seed locks jitter)	AI Logic	PASSED
TC-03	Cascade Delete	Delete Course ID: 101	Course removed from courses.csv and all related ratings removed from ratings.csv	System Logic	PASSED
TC-04	Model Missing	Corrupted or missing students.csv file	System displays a clear error message and halts without data loss	System Logic	PASSED

2. Acceptance Criteria

The system shall be considered accepted for release when all of the following criteria are satisfied:

2.1 Functional Acceptance

- All four test cases (TC-01 through TC-04) must execute with a PASSED result in the target Python environment.
- The recommendation engine must return results only for students who have a recorded rating history; students with no history must receive an empty list accompanied by the message 'No history found'.
- Confidence scores must be applied to all recommendations and remain within the 0.0 to 1.0 range.
- Cascade delete operations must leave no orphaned records in ratings.csv and must produce a log entry verifiable in the audit trail.
- Deletion operations must trigger a warning and require the user to type 'YES' for confirmation.
- All system operations (Add, Update, Delete) must be recorded in system_operations.log immediately.

2.2 Behavioral Acceptance

- Repeated calls to get_recommendations() for the same student within a session must yield identical ranked lists (determinism requirement).
- The DataManager must successfully load students, courses, and ratings from CSV on initialisation without data-loss or encoding errors.
- Invalid menu inputs must trigger the alert: 'Invalid option. Please enter a number from the list.'

2.3 Architectural Acceptance

- The system must respect the three-layer architecture: Presentation / Test Layer calls DataManager (Data Access Layer), and DataManager supplies data to the AI & Business Logic Layer. No layer may directly bypass another.
- SystemLogic must not contain data-file I/O logic directly; all file operations must be delegated to DataManager.

3. Validation Requirements

Validation confirms that the system correctly implements all intended behaviors before deployment.

3.1 Data Validation

#	Validation Point	Method	Pass Condition
V-01	Student exists in dataset	Query DataManager for unknown student ID	Returns empty recommendation list, no exception
V-02	Student Name Integrity	String Pattern Matching	Rejection of names containing numerical digits or special characters (Alphabetic only).
V-03	Cascade delete completeness	Inspect CSV files post delete_course_cascade()	Zero rows referencing deleted course_id remain
V-04	Email Format Integrity	Regular Expression	Rejection of any input that does not follow the name@domain.extension format.

3.2 Integration Validation

#	Validation Point	Method	Pass Condition
I-01	Sequence Flow Integrity	Execution Trace (run_test)	Each step from load_all_data() to print results completes without exceptions.
I-02	End-to-End Activity Flow	Logic Path Testing	Validation of the full pipeline: Data Load → History Check → AI Calculation → Filtering → Display.
I-03	Exception Branching	Edge Case Testing	The 'No history found' branch is successfully triggered and displayed for students without prior ratings.
I-04	Threshold Filtering Logic	Boundary Value Analysis	Verification that courses with Confidence <= 20% are correctly skipped and filtered out of the final list.

4. Performance & Security Testing Requirements

4.1 Performance Requirements

ID	Metric	Target	Test Method
PR-01	Recommendation response time	<= 2 seconds for any single student query	Time get_recommendations() with 1,000+ ratings loaded
PR-02	Data load time	load_all_data() completes in <= 1 second for datasets up to 10,000 rows	Benchmark with synthetic CSV at max expected size
PR-03	Cascade delete throughput	delete_course_cascade() completes in <= 500 ms	Profile with course referenced by 500 ratings
PR-04	Concurrent query stability	System must not crash under 10 simultaneous recommendation requests	Multi-threaded test runner invoking get_recommendations() in parallel

4.2 Security Requirements

ID	Threat / Concern	Requirement	Validation Method
SR-01	Path Traversal	student_name and course_id inputs must be sanitised; no file system path characters (../ \ :) permitted	Inject path traversal strings; confirm rejection
SR-02	CSV Injection	Values written to CSV must be escaped to prevent formula injection (=, +, -, @ prefixes)	Write a crafted student name; open output CSV and confirm no live cell formula
SR-03	Model File Integrity	The .pkl model file must be loaded with pickle only from a trusted, local path; no remote loading permitted	Review DataManager load code; confirm no urllib/requests call is used to fetch model
SR-04	Data Privacy	Student names and rating history must not be written to application logs or error messages in plain text	Inspect all log_event() calls; confirm no PII in log output
SR-05	Error Disclosure	Exceptions must not expose internal file paths, stack traces, or system information to end users	Trigger TC-04 (model missing) and confirm only the approved message string is returned

SR-06	Unauthorized Access	The system shall prompt for an Administrative Password when the 'Admin' role is selected. Access to Admin tools shall only be granted upon successful validation.	Enter an incorrect password; confirm the system denies access. Enter the correct password; confirm access is granted.
-------	---------------------	---	---

4.3 Regression Testing Requirement

After any change to CourseRecommender, SystemLogic, or DataManager, all four test cases (TC-01 to TC-04) and all five validation checks (V-01 to V-05) must be re-executed and passed before the change may be merged. Performance benchmarks PR-01 and PR-02 must also be re-run to detect regressions.